

0. Core Mental Model

FPGA design is concurrent hardware design. HDL syntax is only the notation. The main skill is organizing registers, combinational paths, memories, clocks, resets, interfaces, and verification so the resulting circuit is deterministic and physically realizable.

Software thinking	FPGA thinking
Instructions execute sequentially	Circuits operate concurrently
Variables update immediately	Registers update on clock edges
Loops usually consume time	Loops often describe replicated hardware
Compile success usually means valid code	Synthesis and timing closure mean physically realizable hardware
Debugging is mostly logical	Debugging can be logical, temporal, electrical, or timing-related

- [] Treat every register, memory, and interface as real hardware.
- [] Simulate before flashing to the board.
- [] Read synthesis warnings instead of ignoring them.
- [] Design around clocks, resets, and timing from the start.
- [] Prefer simple, inspectable RTL over clever HDL tricks.

1. Tooling Setup - First 1-2 Weeks

Learn

- FPGA design flow: RTL design, simulation, synthesis, place-and-route, timing analysis, bitstream generation, and programming.
- Difference between simulation behavior, synthesized hardware, and board-level behavior.
- Tang Nano 9K resources: LEDs, buttons, clock source, PLLs, pin constraints, HDMI pins, SDRAM, SPI flash, UART bridge, and expansion headers.
- Constraint files, especially pin assignment and clock definition.

Use Two Tool Flows

Flow	Purpose	Use it when
Gowin EDA	Vendor-supported synthesis, place-and-route, device primitives, timing reports, and programming.	Start here. Avoid fighting toolchain issues before learning FPGA design.
Open-source flow	Yosys, nextpnr-himbaechel, Apicula, and openFPGALoader.	Use later to learn portable flows, scripted builds, and open tooling.

Make

- Blink one LED.
- Blink all LEDs with a counter.
- Make a resettable counter.
- Make a clock-enable-based 1 Hz heartbeat LED.
- Make a simple testbench for the counter.

Checklist

- [] Install Gowin EDA.
- [] Program the Tang Nano 9K successfully.

- [] Build and flash a blink design.
- [] Understand the board constraint file.
- [] Know the board clock frequency.
- [] Create a clean project folder structure.
- [] Use Git from day one.
- [] Write a README for every project.
- [] Add simulation before hardware testing.

Recommended Project Folder Structure

```
fpga-project/
  rtl/
  sim/
  constraints/
  scripts/
  docs/
  build/
  README.md
```

2. Digital Logic Foundations - 4-6 Weeks

Do not skip this stage. FPGA work punishes weak digital fundamentals more severely than software work punishes weak syntax knowledge.

Learn

Area	Topics
Combinational logic	Boolean algebra, multiplexers, decoders, encoders, comparators, adders, priority encoders, barrel shifters.
Sequential logic	D flip-flops, registers, counters, shift registers, FSMs, synchronous reset, asynchronous reset, clock enables.
Number systems	Unsigned arithmetic, signed two's complement, fixed-point arithmetic, overflow, saturation, rounding, truncation, Q-format notation.

Do

- Write a 4-bit adder and an N-bit parameterized adder.
- Write counters: up, down, up/down, saturating.
- Write a shift register, PWM generator, edge detector, debouncer, quadrature decoder, UART transmitter, UART receiver, and simple FSM.

Make

Project	Requirements
Button-controlled LED counter	Button increments a counter; counter is displayed on LEDs; button is debounced; one press equals one count; simulate bounce, reset, normal press, and long press.
PWM dimmer	Generate PWM; use buttons to increase/decrease duty cycle; saturate at 0% and 100%; make frequency configurable.
Quadrature encoder reader	Decode A/B signals; track position; detect direction; handle illegal transitions; simulate forward, reverse, and noisy input.

Checklist

- [] Explain combinational vs sequential logic.
- [] Know why accidental latches are dangerous.
- [] Write clean always_comb and always_ff blocks.
- [] Design an FSM before coding it.
- [] Simulate a design before putting it on hardware.
- [] Read synthesis warnings.
- [] Understand blocking vs nonblocking assignments.
- [] Write parameterized modules.
- [] Explain metastability at a basic level.

3. SystemVerilog for RTL - 1-2 Months

Learn the synthesizable subset deeply. Avoid trying to learn every corner of SystemVerilog early.

Must Know

- module
- logic
- always_comb
- always_ff
- assign
- parameter and localparam
- typedef enum
- packed arrays
- generate blocks
- packages and interfaces later

Avoid Early

- classes
- randomization
- UVM
- unsynthesizable delays
- initial blocks for hardware logic except FPGA-supported memory initialization
- careless for loops
- inferred latches
- multi-clock logic without CDC handling

Basic RTL Template

```
module my_module #(
    parameter int WIDTH = 8
) (
    input  logic          clk,
    input  logic          rst_n,
    input  logic [WIDTH-1:0] in,
    output logic [WIDTH-1:0] out
);

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            out <= '0;
        end else begin
            out <= in;
        end
    end
end

endmodule
```

Make: Reusable Peripheral Library

```
rtl/lib/
debounce.sv
edge_detect.sv
pwm.sv
uart_tx.sv
uart_rx.sv
fifo_sync.sv
pulse_stretcher.sv
reset_sync.sv
quadrature_decoder.sv
```

Checklist

- [] Use logic instead of old-style reg/wire confusion.
- [] Separate combinational and sequential logic.
- [] Avoid unintended latches.
- [] Understand what hardware a for loop creates.
- [] Write parameterized reusable RTL.
- [] Write a testbench for each reusable module.
- [] Explain whether a design infers LUTs, flip-flops, RAM, or DSPs.

4. Simulation and Verification - 2-3 Months

This stage separates hobby-level FPGA work from engineering-level FPGA work. You do not need UVM yet.

Learn

- Testbench structure, clock generation, reset sequencing, self-checking tests, assertions, bounded randomized input, waveform inspection, corner-case testing, regression testing, and functional coverage concepts.
- Tools: Verilator, GTKWave, optional Icarus Verilog, Python test scripting, Makefiles, and cocotb later.

Test Every Design For

- reset behavior
- normal operation
- boundary conditions

- invalid input
- back-to-back transactions
- long-running stability

Make

Project	Requirements
UART loopback with self-checking simulation	UART TX/RX; configurable baud rate; loopback mode; simulation sends bytes and checks received bytes; hardware connects to PC serial terminal.
Synchronous FIFO	Parameterized width/depth; full/empty flags; overflow/underflow protection; self-checking testbench; random push/pop test; hardware demo through UART.

Checklist

- [] Write a self-checking testbench.
- [] Debug with waveforms.
- [] Test reset behavior.
- [] Test boundary conditions.
- [] Run tests from the command line.
- [] Explain what a testbench does not prove.
- [] Stop relying on board-only trial and error.

5. FPGA Timing and Constraints - 2-3 Months

Learn

- Setup time, hold time, clock-to-Q delay, propagation delay, clock skew, slack, maximum frequency, critical path, false paths, multi-cycle paths, clock constraints, generated clocks, input/output delays, and timing closure.
- FPGA fabric: LUTs, flip-flops, carry chains, block RAM, distributed RAM, DSP blocks, PLLs, global clock networks, I/O banks, and I/O standards.

Do

- Revisit old projects and add proper timing constraints.
- Check timing reports and find critical paths.
- Increase clock frequency until timing fails.
- Pipeline the design and compare maximum clock frequency, latency, and resource usage.

Make

Project	Requirements
Pipelined arithmetic unit	Multiply-accumulate operation; fixed-point input/output; unpipelined and pipelined versions; compare max clock frequency, latency, and resource usage.
Fixed-point PID controller	Fixed-point P/I/D terms; saturation; anti-windup; configurable gains; simulation with a simple plant model; hardware output via PWM.

Checklist

- [] Know what setup and hold violations mean.
- [] Read a timing report.

- [] Identify a critical path.
- [] Pipeline a long combinational path.
- [] Explain latency vs throughput.
- [] Constrain a clock.
- [] Prefer clock enables over fabric-generated clocks where appropriate.
- [] Use PLLs deliberately.
- [] Know when to use block RAM.
- [] Know when to use DSP blocks.

6. Real Interfaces - 3-6 Months

This stage makes the FPGA useful in real mechatronics and embedded systems projects.

Learn

Class	Interfaces / Concepts
Low-speed interfaces	UART, SPI, I2C, GPIO, PWM, quadrature encoder, one-wire-style protocols.
Medium-speed interfaces	VGA-style timing, HDMI basics, SDRAM controller basics, parallel RGB LCD, audio I2S, camera parallel interface later.
Protocol concepts	Valid/ready handshaking, FIFOs, buffering, framing, packetization, CRC, timeout handling, bus arbitration.

Make

Project	Requirements
SPI sensor interface	SPI master; configurable clock divider; chip-select timing; receive data; convert raw data to fixed-point; stream result over UART.
I2C sensor reader	I2C master; start/stop generation; ACK/NACK handling; register read/write; timeout handling; UART debug output.
Motor-control peripheral set	PWM generator; encoder reader; speed estimator; ADC interface if available; fault input; emergency stop; UART command interface.
HDMI or LCD graphics demo	Generate timing; framebuffer or procedural graphics; draw simple shapes; animate object; show sensor value or counter onscreen.

Checklist

- [] Design a peripheral from a timing diagram.
- [] Read a datasheet and convert it into an FSM.
- [] Build SPI master logic.
- [] Build I2C master logic.
- [] Build a UART debug interface.
- [] Understand valid/ready handshakes.
- [] Use FIFOs between producer and consumer logic.
- [] Debug hardware with internal signal capture or an external logic analyzer.
- [] Design with timeouts and error states.

7. Memory Systems - 2-4 Months

Learn

- Registers vs distributed RAM vs block RAM.
- Single-port RAM, simple dual-port RAM, true dual-port RAM, and ROM initialization.
- FIFOs, circular buffers, line buffers, SDRAM concepts, memory latency, arbitration, burst access, and cache basics later.

Make

Project	Requirements
Block RAM framebuffer	Store pixel data in BRAM; read pixels for display; write pixels from another process; handle read/write conflicts; display simple graphics.
Logic analyzer core	Sample internal signals into RAM; trigger condition; pre-trigger/post-trigger capture; UART dump to PC; Python script to display waveform.
SDRAM experiment	Initialize SDRAM; write/read test patterns; verify data; expose memory test result via UART or display; measure effective throughput.

Checklist

- ☐ Know when flip-flop storage is wasteful.
- ☐ Infer block RAM.
- ☐ Build a FIFO.
- ☐ Understand memory read latency.
- ☐ Design around RAM latency.
- ☐ Arbitrate between multiple memory clients.
- ☐ Test memory with walking-ones and walking-zeroes patterns.
- ☐ Understand why SDRAM controllers are nontrivial.

8. Clock Domain Crossing and Reliability - 1-2 Months

Learn

- Metastability, synchronizers, two-flop synchronizer, pulse crossing, toggle synchronizer, asynchronous FIFO, Gray-coded pointers, reset domain crossing, debounce vs synchronization, external asynchronous inputs, and clock enables vs generated clocks.

Make

Project	Requirements
CDC demonstration	Two independent clocks; unsafe crossing version; safe synchronizer version; observable failure or explanation; simulation plus hardware test.
Asynchronous FIFO	Write clock; read clock; Gray-code pointers; full/empty flags; self-checking testbench; stress test with unrelated clocks.

Checklist

- ☐ Never feed external inputs directly into large logic.
- ☐ Synchronize asynchronous inputs.

- [] Understand that simulation may not show metastability.
- [] Cross a single-bit level signal safely.
- [] Cross a pulse safely.
- [] Cross a multi-bit bus safely.
- [] Design or use an asynchronous FIFO.
- [] Understand reset synchronization.

9. DSP and Control on FPGA - 3-6 Months

This is the most directly relevant specialization for mechatronics.

Learn

- Fixed-point arithmetic, saturation arithmetic, rounding, scaling, FIR filters, IIR filters, moving average filters, CORDIC, numerically controlled oscillators, PWM modulation, PID control, motor commutation basics, encoder velocity estimation, sensor fusion basics, latency budgeting, and sample-rate design.

Make

Project	Requirements
Fixed-point FIR filter	Parameterized taps; fixed-point coefficients; impulse response test; step response test; Python-generated test vectors; compare RTL output against Python model.
CORDIC sine/cosine	Fixed-point angle input; sine/cosine output; pipelined architecture; error measurement; compare against Python reference.
FPGA-based motor controller	PWM generation; encoder feedback; velocity estimator; PID speed loop; UART command interface; safety cutoff; hardware-in-the-loop testing.
Sensor acquisition pipeline	SPI/I2C sensor input; filtering; scaling; timestamping; buffering; UART/USB output; optional display output.

Checklist

- [] Choose a fixed-point format.
- [] Calculate overflow bounds.
- [] Pipeline DSP logic.
- [] Use FPGA multipliers/DSP blocks.
- [] Compare RTL against a Python or MATLAB model.
- [] Design a real-time control loop.
- [] Budget latency through a control pipeline.
- [] Explain sample rate vs clock rate.

10. Soft CPUs and FPGA SoC-Lite - 4-8 Months

Do this after you are comfortable with RTL, memory, timing, and interfaces. A soft CPU should not be used to hide weak hardware design fundamentals.

Learn

- CPU datapath, instruction fetch/decode/execute, register file, ALU, load/store, memory-mapped I/O, interrupts, bus interconnects, RISC-V basics, firmware/toolchain basics, linker scripts later, boot ROM concept, and debug interface concept.

Make

Project	Requirements
Tiny custom CPU	8-bit or 16-bit instruction width; small instruction ROM; register file; ALU; branch/jump; memory-mapped LED/PWM/UART; assembler script in Python.
Minimal RISC-V soft-core integration	Instantiate PicoRV32, FemtoRV, SERV, or NeoRV32; map GPIO, UART, timer, and PWM; write bare-metal C; compile and load firmware.
Hardware accelerator attached to soft CPU	CPU writes input data to accelerator registers; accelerator processes data; CPU polls or receives interrupt; result returned through registers; compare software vs hardware execution.

Good Accelerator Examples

- FIR filter
- PID block
- CORDIC
- encoder processing
- CRC engine
- small matrix multiply
- stepper trajectory generator

Checklist

- [] Understand memory-mapped I/O.
- [] Separate hardware control from firmware control.
- [] Integrate a soft CPU.
- [] Write simple bare-metal C.
- [] Expose FPGA peripherals to software.
- [] Design a register map.
- [] Document a hardware/software interface.
- [] Understand why SoC design is mostly integration and verification.

11. FPGA Architecture and Professional Practice - Ongoing

Learn

- FPGA LUT architecture, carry chains, DSP slices, BRAM architecture, clock routing, I/O timing, SERDES basics, configuration bitstreams, partial reconfiguration conceptually, vendor IP, portability issues, floorplanning later, power estimation, and design-for-debug.

Engineering Deliverables for Each Nontrivial Project

1. Requirements
2. Block diagram
3. Clock/reset plan
4. Interface definitions
5. Register map if applicable
6. Timing constraints
7. Verification plan

8. Resource report
9. Timing report
10. Known limitations

Checklist

- [] Write specs before RTL.
- [] Draw block diagrams.
- [] Define interfaces before coding internals.
- [] Verify modules independently.
- [] Inspect warnings.
- [] Review timing reports.
- [] Document constraints.
- [] Version-control everything.
- [] Reproduce a build from a clean checkout.
- [] Explain resource usage and timing bottlenecks.

12. Capstone Projects

Complete at least two substantial FPGA projects before moving seriously toward ASIC/SoC work.

Capstone	Features	What it proves
Mechatronics Control FPGA	UART command interface; PWM motor output; quadrature encoder input; fixed-point PID; safety fault handling; telemetry stream; optional display; optional soft CPU supervisor.	Real-time control, fixed-point arithmetic, peripheral design, hardware debugging, and mechatronics relevance.
FPGA Mini-SoC	RISC-V soft CPU; ROM/RAM; UART; GPIO; timer; PWM; custom accelerator; memory-mapped register bus; bare-metal firmware.	CPU integration, hardware/software boundary design, bus design, firmware interaction, and SoC architecture foundation.
Video or Signal System	Input stream; line buffers; filter pipeline; video/display output; UART control; fixed-point processing; timing/resource analysis.	Streaming architecture, pipelining, memory bandwidth awareness, real-time processing, and interface timing.

13. When to Upgrade Beyond Tang Nano 9K

Do not upgrade too early. Exhaust the Tang Nano 9K with disciplined projects first. Upgrade when the board becomes the bottleneck, not when the projects become difficult.

Upgrade When You Hit Limits In

- block RAM
- DSP count
- SDRAM bandwidth
- CPU/SoC complexity
- video buffering
- Ethernet
- high-speed I/O
- larger RISC-V cores

- Linux-capable SoCs

Stage	Board class
Next FPGA board	Tang Nano 20K / Tang Primer 20K / ECP5 board
Serious FPGA learning	Artix-7 / ECP5 / Cyclone-class board
Embedded Linux + FPGA	Zynq / SoC FPGA board
ASIC-adjacent open-source flow	TinyTapeout / Sky130 / IHP130 experiments

14. Later Path: ASIC and SoC Preparation

ASIC and full SoC work should come after you can already build reliable FPGA systems. FPGA competence is the prerequisite, not the side quest.

Before ASIC, Learn

- synchronous digital design deeply
- static timing analysis
- verification
- SystemVerilog assertions
- clock/reset design
- CDC
- linting
- synthesis constraints
- scan/DFT basics
- power/area/timing tradeoffs
- bus protocols
- CPU architecture basics
- physical design basics

ASIC-Specific Topics

Area	Topics
Front-end ASIC	RTL design, lint, CDC/RDC checks, synthesis, formal verification, assertions, constrained-random verification, UVM, low-power design, clock gating, reset strategy, DFT/scan awareness.
Back-end ASIC	Standard cells, floorplanning, power grid, placement, clock tree synthesis, routing, parasitics, static timing analysis, IR drop, electromigration, signoff, GDSII.
SoC topics	AMBA APB/AHB/AXI, Wishbone, TileLink later, DMA, caches, interrupts, privilege levels, boot flow, memory maps, debug modules, firmware/driver boundary, hardware security basics.

15. Suggested Timeline

Time	Focus	Output
Month 1	Tooling, Verilog/SystemVerilog basics	LED, counter, PWM, button debounce
Months 2-3	Simulation, FSMs, UART, reusable modules	UART loopback, FIFO, encoder reader
Months 4-5	Timing, constraints, pipelining	Pipelined arithmetic, fixed-point PID
Months 6-8	Interfaces	SPI/I2C sensors, motor-control peripherals
Months 9-11	Memory/video	BRAM framebuffer, SDRAM test, HDMI/LCD demo

Time	Focus	Output
Months 12-15	CDC, robust systems	Async FIFO, multi-clock design
Months 16-20	DSP/control	FIR, CORDIC, FPGA control loop
Months 21-28	Soft CPU / SoC-lite	RISC-V soft-core with custom peripherals
Months 29-36	Larger capstone	Mechatronics FPGA controller or mini-SoC
Year 3+	ASIC/SoC foundations	Formal, UVM, STA, OpenROAD, tapeout experiments

16. Master Checklist

Fundamentals

- ☐ Boolean logic
- ☐ combinational logic
- ☐ sequential logic
- ☐ FSMs
- ☐ binary arithmetic
- ☐ signed arithmetic
- ☐ fixed-point arithmetic
- ☐ pipelining
- ☐ resource sharing
- ☐ latency vs throughput

HDL

- ☐ synthesizable SystemVerilog
- ☐ blocking vs nonblocking assignments
- ☐ parameterized modules
- ☐ clean reset strategy
- ☐ no inferred latches
- ☐ clean FSM style
- ☐ testbench writing
- ☐ basic assertions
- ☐ basic linting

FPGA implementation

- ☐ Gowin EDA flow
- ☐ constraints file
- ☐ pin planning
- ☐ synthesis report
- ☐ place-and-route report
- ☐ timing report
- ☐ resource usage report
- ☐ PLL use
- ☐ BRAM inference
- ☐ DSP inference

Interfaces

- ☐ GPIO
- ☐ button debounce
- ☐ UART
- ☐ SPI
- ☐ I2C
- ☐ PWM
- ☐ quadrature encoder
- ☐ LCD/HDMI timing
- ☐ SDRAM basics
- ☐ memory-mapped registers

Verification

- ☐ self-checking testbenches
- ☐ waveform debugging
- ☐ randomized tests
- ☐ Python reference models
- ☐ regression scripts
- ☐ assertions
- ☐ hardware debug signals
- ☐ logic-analyzer-style capture

Timing/reliability

- ☐ setup/hold timing
- ☐ slack
- ☐ critical path
- ☐ clock constraints
- ☐ generated clocks
- ☐ false paths
- ☐ multi-cycle paths
- ☐ CDC
- ☐ reset synchronization
- ☐ asynchronous FIFO

Mechatronics FPGA applications

- ☐ PWM motor control
- ☐ encoder feedback
- ☐ fixed-point PID
- ☐ sensor acquisition
- ☐ filtering
- ☐ trajectory generation
- ☐ safety interlocks
- ☐ real-time telemetry
- ☐ FPGA + microcontroller partitioning

SoC preparation

- ☐ custom simple CPU
- ☐ RISC-V basics
- ☐ soft-core integration
- ☐ memory map
- ☐ bus interconnect
- ☐ interrupt handling
- ☐ firmware build flow
- ☐ hardware accelerator interface
- ☐ boot ROM concept

ASIC preparation

- ☐ static timing analysis
- ☐ formal basics
- ☐ SystemVerilog assertions
- ☐ lint
- ☐ CDC/RDC analysis
- ☐ synthesis constraints
- ☐ standard-cell concepts
- ☐ floorplanning basics
- ☐ place and route basics
- ☐ OpenROAD-style flow basics

17. Recommended Project Order

1. LED blink
2. LED counter
3. button debounce
4. PWM dimmer
5. quadrature decoder
6. UART TX
7. UART RX
8. UART loopback
9. synchronous FIFO
10. SPI master
11. I2C master
12. fixed-point arithmetic library
13. FIR filter
14. CORDIC
15. PID controller
16. motor-control peripheral block
17. BRAM framebuffer

18. HDMI/LCD graphics demo
19. SDRAM memory test
20. logic analyzer core
21. async FIFO
22. tiny custom CPU
23. RISC-V soft-core SoC
24. custom accelerator
25. mechatronics FPGA capstone
26. mini-SoC capstone

18. What Not to Waste Time On Early

These are not bad topics. They are bad early topics because they can hide weak fundamentals or consume large amounts of setup/debug time before you have the required base.

- UVM
- HLS
- Linux-on-FPGA
- AXI-heavy designs
- PCIe
- DDR3/DDR4 controllers
- Ethernet MAC/PHY
- complex out-of-order CPUs
- ASIC physical design
- formal verification beyond small properties
- AI accelerators without DSP and memory fundamentals

19. Immediate 30-Day Plan

Week	Tasks	Expected output
Week 1	Install Gowin EDA; program Tang Nano 9K; blink LED; create Git repo; learn constraint basics; write first testbench.	Working board flow and a reproducible blink project.
Week 2	Counter on LEDs; button debounce; edge detector; simulate button bounce; write clean README.	Reliable human-input handling.
Week 3	PWM generator; button-controlled duty cycle; parameterize PWM resolution; verify PWM frequency; inspect synthesis report.	Reusable PWM peripheral.
Week 4	UART transmitter; UART receiver; UART loopback; PC terminal communication; self-checking UART simulation.	Basic debug and communication channel.

At the end of 30 days, the base library should contain:

```
debounce.sv
edge_detect.sv
pwm.sv
uart_tx.sv
uart_rx.sv
counter.sv
reset_sync.sv
```

References and Starting Links

These are practical starting points for the Tang Nano 9K board, Gowin tooling, and open-source Gowin flows.

- Sipeed Tang Nano 9K hardware documentation: <https://wiki.sipeed.com/hardware/en/tang/Tang-Nano-9K/Nano-9K.html>
- Sipeed Tang Nano 9K examples and project notes: <https://wiki.sipeed.com/hardware/en/tang/Tang-Nano-9K/Tang-nano-9k.html>
- Gowin EDA support page: <https://www.gowinsemi.com/en/support/home/>
- Sipeed Tang IDE installation documentation: <https://wiki.sipeed.com/hardware/en/tang/Tang-Nano-Doc/install-the-ide.html>
- Sipeed Linux flashing documentation: <https://wiki.sipeed.com/hardware/en/tang/Tang-Nano-Doc/flash-in-linux.html>
- Project Apicula for Gowin open-source tooling: <https://github.com/YosysHQ/apicula>
- OpenROAD/Yosys-style open-source SoC tapeout reference: <https://arxiv.org/abs/2405.04257>